Tectonic: Carrying WireGuard over Short-Lived, Unordered TLS Lanes

The PangeaVPN Team

PangeaVPN

Technical Report · Version 1.0 · November 2026

Abstract

Tectonic is PangeaVPN's TCP transport for WireGuard. It spreads WireGuard datagrams across a small pool of short-lived TLS 1.3 connections, called *lanes*, and puts them back in WireGuard's own order on arrival. Each lane has its own congestion window, so a loss slows one lane and the others keep their pace. To an observer, each lane is Chrome loading an ordinary website, down to the size of the first records it sends. A channel-bound login lets a node accept any server name or none, and any connection that fails the login is served the node's real website. Tectonic adds no cryptography of its own.

We describe the protocol and evaluate it against single-connection designs (REALITY and AnyTLS) and plain WireGuard over UDP in Linux network namespaces. On lossy paths with a cubic node, Tectonic carries 2.2–3.4× the throughput of single-connection designs. On clean links it matches them, at 95–97% of plain WireGuard. The first packet goes out within two round trips, and 19–20 of 20 active probes are answered exactly as a stock Go web server answers them.

Keywords: censorship circumvention, WireGuard, TLS 1.3, multipath transport, traffic shaping, active probing.

Status: the protocol is implemented and covered by tests, including an end-to-end run with real WireGuard, but it has not been tested on real networks. All measurements in this report come from a lab, and the design does not claim to be faster until field tests confirm them.

1 Introduction

Censoring networks identify VPN traffic by its shape, and they degrade TCP-based disguises by degrading TCP itself. A circumvention transport has to fix both at once.

WireGuard is fast, simple and well audited, but its packets are easy to recognise: a 148-byte handshake initiation, a 92-byte response, a 32-byte keepalive on a fixed timer, and UDP throughout. Many networks block UDP outright or classify it on sight. The usual answer is to wrap the VPN in TLS over a single TCP connection, as REALITY, Trojan and AnyTLS do. That fixes how the traffic looks and creates three new weaknesses.

Head-of-line blocking

One TCP connection delivers bytes in order. One lost segment holds up every packet behind it, and the connection's single congestion window halves for all traffic at once. At 1% loss on a 100 Mbit/s, 50 ms path, a single connection carries about 2.5 Mbit/s.

Long-lived, conspicuous flows

A tunnel that holds one connection open for hours, with steady throughput and fixed-period keepalives, looks nothing like a browser. Networks use this to throttle long connections or reset them.

Per-connection throttling
Some networks cap each TCP connection at a few Mbit/s. A single-connection transport cannot get past that cap however good its disguise is.

Server-name filtering
SNI allowlists, blocklists and active probes all target the TLS handshake. A design that accepts only one server name, or answers probes in its own way, can be identified from that.

Tectonic starts from one observation: **the payload is always WireGuard datagrams, never a byte stream.** WireGuard already encrypts, authenticates, numbers and de-duplicates every packet, so the transport doesn't have to keep a strict order across connections. That allows multiple independent lanes, free duplication of urgent packets, connections that turn over the way a browser's do, and a session that outlives every one of its connections.

1.1 The core ideas

#	Idea	What it does
01	**Unordered lanes**	Datagrams spread over several TLS lanes, each with its own congestion window. On a lossy path the lanes together carry far more than one connection.
02	**Free deduplication**	WireGuard's anti-replay window already drops duplicates, so small urgent packets can be sent on two lanes with no bookkeeping on the receiving end.
03	**Lane churn**	Lanes live from tens of seconds to a few minutes, and a replacement opens before the old lane closes. Time sets how often lanes turn over, not throughput.
04	**Throttle detection**	If adding a lane adds throughput while each lane stays capped, the network is throttling each connection, and Tectonic adds lanes, up to a limit.
05	**Our own identity**	Each node serves a real website on a domain PangeaVPN owns, with a real certificate. SNI, IP address and certificate all agree.
06	**Channel-bound login**	The login is bound to its TLS session through a TLS exporter, so a captured login is useless on any other connection.
07	**Shaping budget**	Padding is spent where WireGuard would otherwise show: handshakes, keepalives, idle periods and small packets. Bulk transfer takes almost none.
08	**Sessions outlive connections**	A session belongs to the device. Every lane can drop during a network change, and new lanes rejoin the same session.
09	**Any SNI, or none**	The login happens inside TLS, so the node accepts whatever server name the network allows, or no name at all.

2 Design goals and non-goals

Goals

- **Fast.** Fast enough to be the default transport on ordinary networks. The first packet flows within two round trips, throughput stays close to plain WireGuard, and loss hurts it far less than it hurts single-connection TCP transports.
- **Unremarkable on the wire.** Every lane looks like Chrome talking to an ordinary HTTPS server.
- **Resilient.** It survives injected resets, per-connection throttling, long-connection

Non-goals

- **IP allowlists.** No protocol gets through a network that admits only approved IP addresses. SNI allowlists are a different matter and are covered in §09.
- **Secrecy.** The code is public under GPL-3.0, and nothing in the design depends on a censor not knowing how it works.
- **Replacing WireGuard.** Tectonic only carries WireGuard, which still provides the end-to-end encryption and keys.

heuristics, SNI allowlists, and a phone moving between Wi-Fi and mobile data.

- **One protocol, several routes.** The same core runs direct to the node or through a CDN.
- **No home-made cryptography.** Real TLS 1.3 outside and WireGuard inside. Tectonic adds only authentication, framing, scheduling and shaping, built from HMAC-SHA256, HKDF and TLS exporters.

PLACE IN THE CASCADE Tectonic is one transport in PangeaVPN's transport cascade, not a replacement for it. Shaping makes classification harder and more expensive, but cannot rule out a well-resourced censor running machine-learning classifiers over whole flows. REALITY, the CDN carrier and the other transports stay in place behind it.

3 Architecture

On the device, WireGuard sends to a loopback UDP port, as it does with every other PangeaVPN transport. The Tectonic client spreads those packets across its lanes. The node merges what arrives back into the order WireGuard sent it and passes it to the node's WireGuard. To the PangeaVPN daemon, Tectonic is one more transport: a loopback UDP listener that WireGuard's endpoint points at.

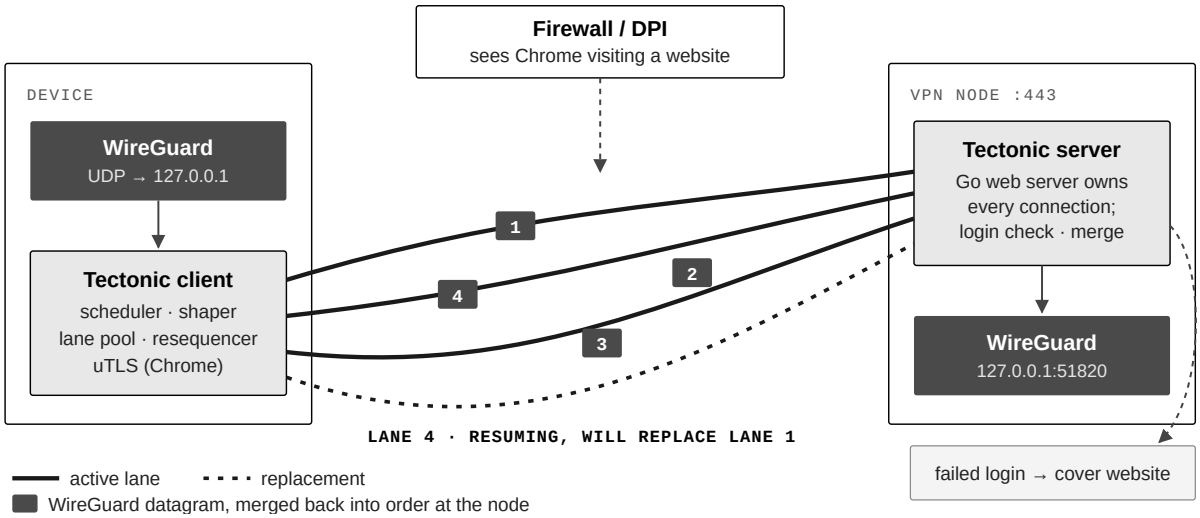


Figure 1. System overview. Datagrams cross several TLS lanes with no order shared between them, and the receiver merges them back into WireGuard's counter order. A replacement lane opens before the lane it replaces closes. Any connection that fails the login stays with the cover website's web server.

3.1 Layering

Tectonic sits between WireGuard and TLS. Everything it adds (login, frames, scheduling and shaping) is inside the TLS record layer, where a passive observer can measure only record sizes and timing.

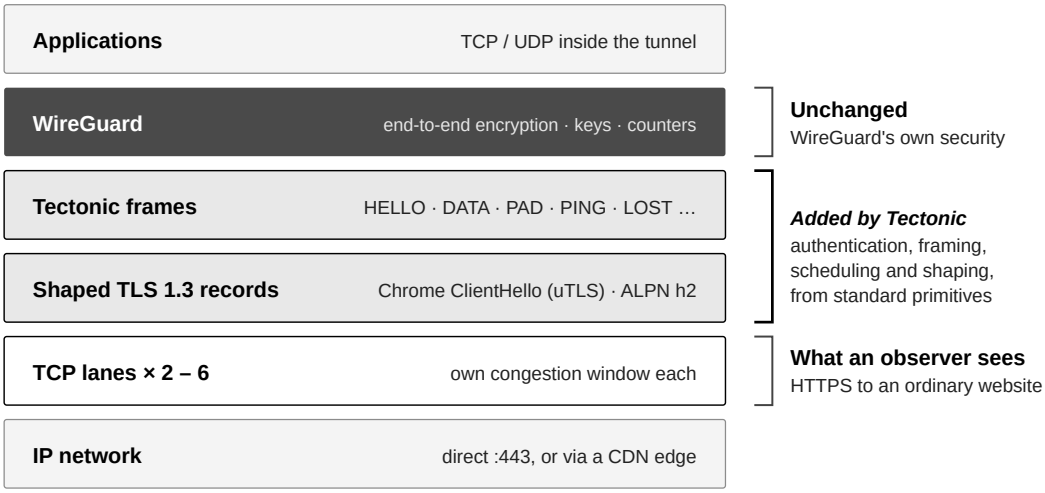


Figure 2. Protocol stack. Tectonic invents no cryptography. WireGuard keeps end-to-end confidentiality and integrity, and TLS 1.3 supplies the outer disguise and the channel binding for the login.

4 Connecting, step by step

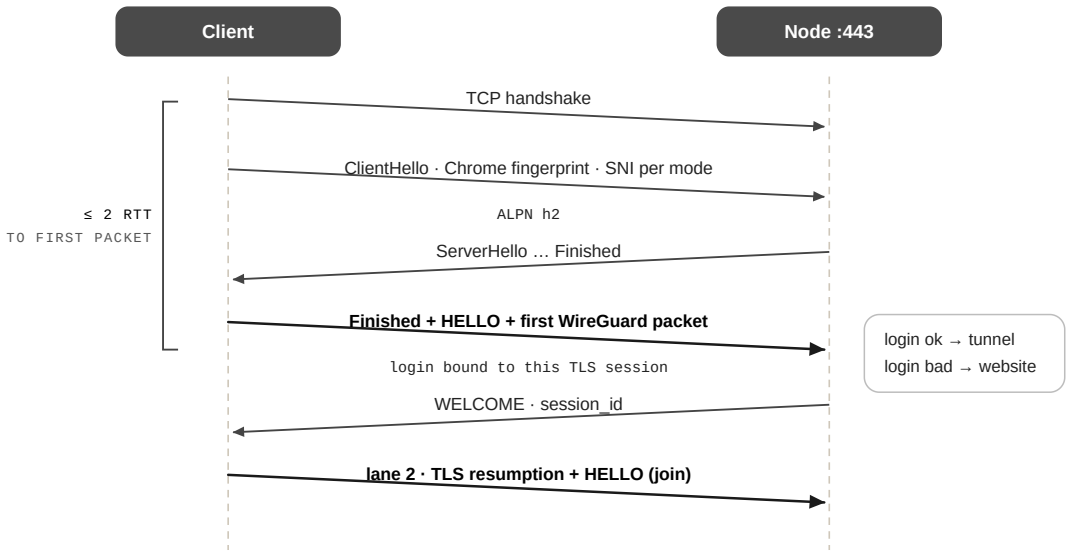


Figure 3. Lane establishment. The login and the first WireGuard packet go out together right after the TLS handshake, without waiting for a reply. Time to first packet is one TCP round trip plus one TLS round trip.

1. **Provisioning.** The hub returns a `tectonic` profile: node address, cover domain, `key_id`, `device_key` and an optional CDN route.
2. **First lane.** TCP to the node, then a TLS 1.3 handshake with Chrome's ClientHello via uTLS, offering ALPN `h2`. The SNI depends on the SNI mode.
3. **Login in the first flight.** `HELLO` and the first WireGuard packet go out in the two records Chrome opens HTTP/2 with: 87 bytes, then a request-sized record.
4. **The node decides.** The cover site's web server owns the connection from accept. A valid `HELLO` takes it for a lane, and the node answers `WELCOME`. Anything else stays with the web server.
5. **More lanes.** A second lane joins at once using TLS session resumption, the way a returning browser reconnects.
6. **Network change.** If every lane dies, the client rejoins with the same `session_id` within the resume window (60 s by default). WireGuard's session carries on without noticing.

5 Authentication

Nodes keep no device database. A credential proves itself, and a login is good only on the TLS connection it was made for.

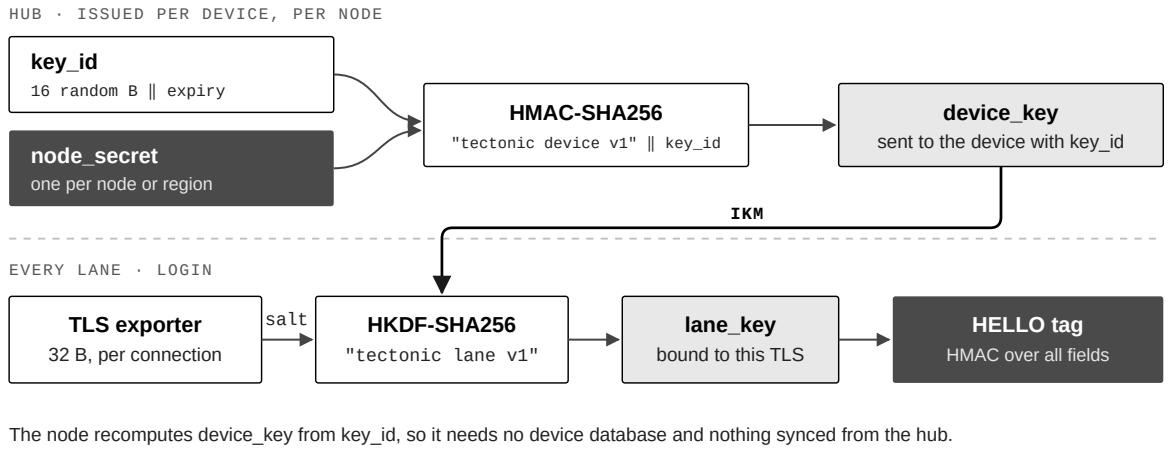


Figure 4. Credential and login derivation. A `key_id` cannot be forged, and its expiry cannot be extended, without `node_secret`. The TLS exporter ties every login to one TLS connection.

```
key_id     = 16 random bytes ‖ expiry (uint32, unix days)            // 20 bytes
device_key = HMAC-SHA256(node_secret, "tectonic device v1" ‖ key_id)
exporter   = TLS-Exporter("EXPORTER-tectonic-hello", context = "", 32 bytes)
lane_key   = HKDF-SHA256(ikm = device_key, salt = exporter, info = "tectonic lane v1")
```

5.1 Properties

- **Channel binding.** The exporter value is unique to each TLS connection, so a `HELLO` copied from one connection fails on every other. A ±120 s timestamp window is a second defence against replay.
- **Session joining** requires the `key_id` that created the session, so a leaked `session_id` is useless on its own.
- **Constant-time checks.** Success and failure take the same time. The check runs on every first record, whatever it contains, so its timing reveals nothing about what the client sent.
- **Revocation.** The hub reissues credentials before they expire, and revokes early by pushing a short deny-list of `key_id`s, which the node reloads on SIGHUP.

5.2 When the login fails

The node never sends a Tectonic-specific error, and it never waits where a web server wouldn't. The cover website's Go web server owns every connection from the moment it is accepted. A client that doesn't log in gets what that server gives anyone: its HTTP/2 `SETTINGS` frame first, the site for a valid request, an ordinary `400 Bad Request` for anything else, one timeout if it stalls, and a TLS close alert on hang-up. Built with Go 1.27 or newer, the node serves a wrapped connection exactly as Go serves one it accepted itself.

6 Wire format

Inside TLS, a lane carries a series of length-prefixed frames. Several frames can share one TLS record, a frame can begin in one record and end in the next, and the shaper decides how large each record is. On a

direct lane, the first bytes from the server are the web server's own HTTP/2 `SETTINGS` frame. The client reads past it.

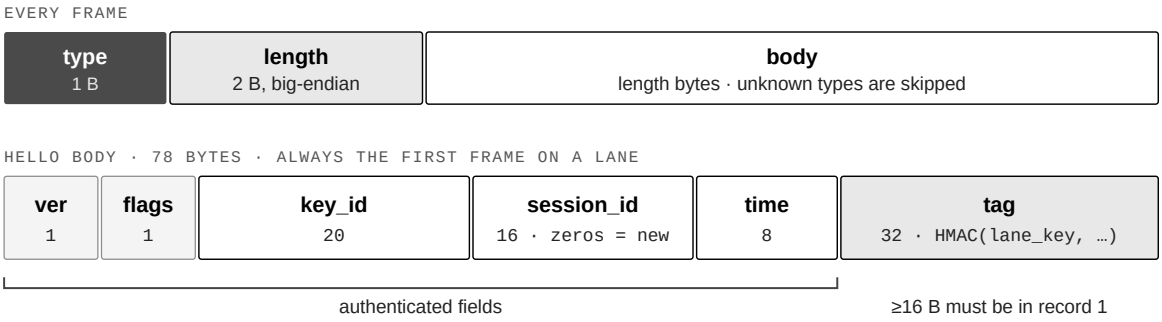


Figure 5. Frame and HELLO layout. The first client record must hold all of HELLO's fields and at least 16 bytes of its tag. The rest of the tag may follow in the next record, and the node waits for it only if the part that arrived is correct.

Type	Name	Dir.	Body
`0x01`	**HELLO**	C → S	The login above.
`0x02`	**WELCOME**	S → C	`session_id`, `max_lanes`, `resume_window`, shaping profile ID
`0x03`	**DATA**	both	Exactly one WireGuard datagram. No sequence number: WireGuard's own counters handle order and duplicates.
`0x04`	**PAD**	both	Zeros, discarded on receipt.
`0x05`	**PING / PONG**	both	An 8-byte token for per-lane round-trip time.
`0x06`	**RETIRE**	both	No new DATA on this lane. It drains, then closes with TLS `close_notify`.
`0x07`	**CLOSE**	both	Ends the whole session, with a 1-byte reason.
`0x08`	**LOST**	both	A WireGuard key index, counter and count: datagrams the sender dropped and won't send. The receiver stops waiting for them.

7 Lanes and the scheduler

7.1 Choosing a lane

For each outgoing datagram, the scheduler estimates when each lane would deliver it:

```
eta(lane) = queued_bytes(lane) / rate(lane) + srtt(lane) / 2
```

Lanes are kept in order of preference, newest first. A packet goes on the most preferred lane whose estimate is within a sixteenth of that lane's round trip of the earliest. Traffic that one lane can carry stays on that lane, in order, and spills over to the next only once the first falls behind.

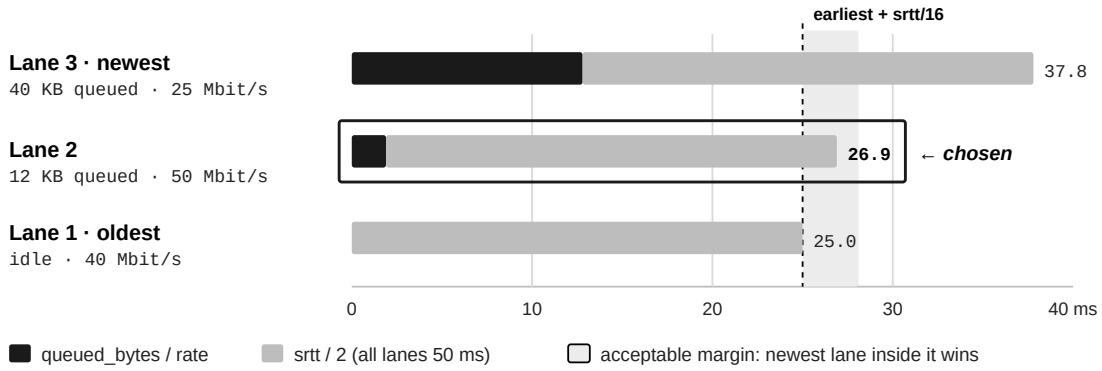


Figure 6. Choosing a lane (illustrative numbers). Lane 1 would deliver earliest, at 25 ms. Lane 2 is within the margin and preferred over lane 1, so it takes the packet. The newest lane is preferred but has too much queued. Preferring newer lanes is how a replacement lane takes over traffic during churn.

- **Shallow socket buffers.** Each lane sets `TCP_NOTSENT_LOWAT`, so the queue stays in Tectonic's scheduler, which can still steer it, rather than in the kernel.
- **What the kernel holds counts.** Where `TCP_INFO` exists, `queued_bytes` includes unsent kernel bytes, and `rate` is the congestion window per round trip or the pacing rate. Without it (Windows), rate and RTT come from PING/PONG, and the lane limits its own outstanding bytes and paces itself.
- **Records no larger than a moment of sending.** Nothing in a TLS record is usable until all of it arrives, so a record holds at most a thirty-second of the lane's round trip of data: a full 16 KB at 100 Mbit/s, and a single packet on a freshly opened paced connection.
- **Small packets first.** Packets of 128 bytes or less overtake bulk data queued on the same lane, so pings and ACKs don't wait behind a download.
- **A full queue drops.** Once every lane's oldest packet has waited about a round trip, new packets are dropped, which the inner TCP reads as a full link. The drop is reported to the receiver in a `LOST` frame.
- **Session send limit.** When the lanes' congestion control doesn't back off (BBR's start-up phase), a shared limit engages once retransmissions exceed 5% while RTT rises in two consecutive two-RTT windows. It then tracks the delivered rate, and stands down on paths it doesn't suit.

7.2 Putting packets back in order

Lanes are separate TCP connections, so packets arrive out of order across them. A TCP connection inside the tunnel reads a late packet as a lost one: it retransmits and slows down for a loss that never happened. The receiver therefore merges the lanes back into one ordered stream, **with no sequence numbers of its own**. The counter in WireGuard's cleartext data header rises by one per packet, and that is enough.

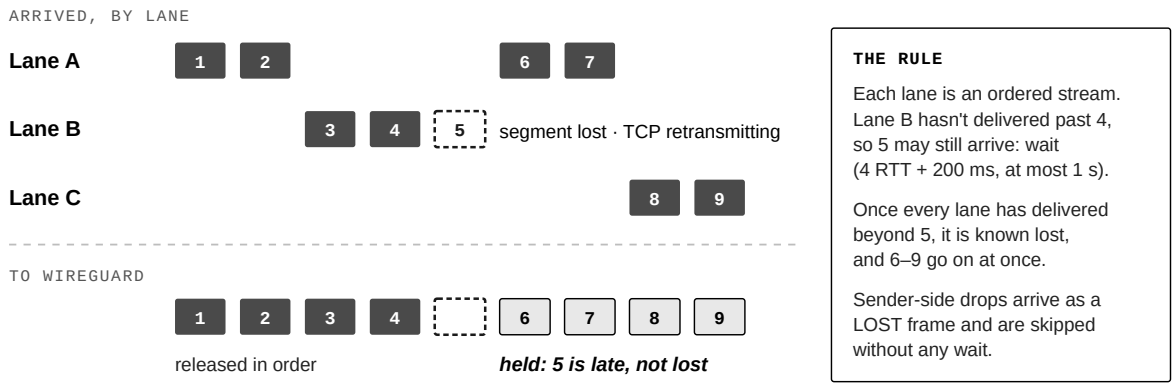


Figure 7. Resequencing by WireGuard counter. No timer guesses whether a packet is late. A packet is lost only when no lane can still deliver it. Packets of 128 bytes or less, and anything that isn't WireGuard data, bypass the merge.

Why the wait is long Handing on packets past a gap that is still in flight looks like loss to the inner TCP, which then resends the whole stretch and slows down. The first version waited 1.5 round trips. On Linux TCP it gave up on hundreds of packets a run, nearly all of which then arrived, and carried 27–33 Mbit/s on a 1%-loss path with a BBR node. With a wait of four round trips plus 200 ms, the same path carries about 79 Mbit/s. A lane that was already quiet before the gap gets a short wait of 1.5 round trips instead, so a lane that died with the packet doesn't stall the tunnel.

7.3 Duplicating small urgent packets

While any lane is retransmitting, packets of 128 bytes or less (WireGuard handshakes, keepalives, most inner TCP ACKs) are also copied onto the next-best lane, capped at 5% of bandwidth. The receiver needs no logic for this, because WireGuard's anti-replay window drops the second copy.

7.4 Lane churn

No connection runs for hours. Each lane's lifetime is drawn from a browser-like distribution (median about 90 s, range 20 s to 6 min), and its replacement opens with TLS resumption shortly before it expires. As the newest lane, the replacement takes traffic as fast as its connection can grow. The old lane is then sent `RETIRE`, finishes sending, and closes. The number of working lanes never drops during the switch.



Figure 8. Time-driven lane churn. Lifetimes are random and set by the clock, not by bytes carried. A download that opened a fresh connection every few seconds would mark the client far more plainly than anything one connection carries. Each lane also has a byte budget (about 2 GiB median), which rarely applies below about 190 Mbit/s.

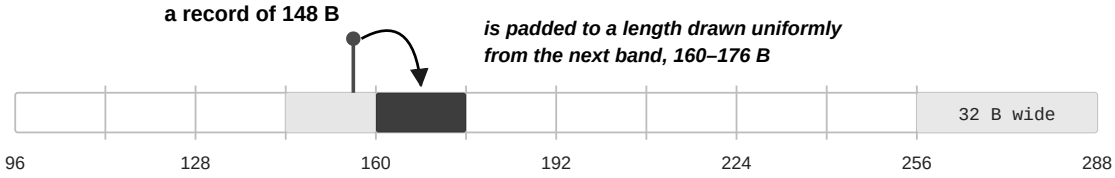
7.5 Lane count and throttle detection

- The pool starts with 2 lanes and can grow to `max_lanes`, which the server sets: 6 by default, 3 on cellular.
- **Throttle check.** Under load, if every lane settles at about the same rate and adding a lane adds about that much again, the network is limiting each connection separately. The client keeps adding lanes while each one pays off, and closes them once they stop helping. Under a 4 Mbit/s per-connection cap, Tectonic reached 19–24 Mbit/s on Linux with a cubic node.
- **Idle.** When the tunnel goes idle, the pool shrinks to one lane, at a moment drawn for each idle period (20 to 140 s after the last traffic with the default profile), so connections don't close at the same offset in every session.
- **Resets.** If the network kills a lane, for example with a forged RST or a middlebox timeout, traffic moves to the other lanes at once and a replacement opens. In the lab, the longest stall inside the tunnel after a lane reset was 76–139 ms.

8 Traffic shaping

TLS hides what traffic contains, but WireGuard still has a recognisable shape. Tectonic changes only what an observer can measure from outside: TLS record sizes and their timing.

State	What Tectonic does	Overhead
Bulk	While more is waiting than one record holds, fills every record to the full 16 KB, as a real download does, and lets a packet span two records. A record that would end between packets ends instead at a point drawn evenly inside its last packet, and the rest follows at once, so record lengths don't fall on a lattice of packet sizes.	**≈ 0–3%**
Interactive	Gives a record up to 1 ms to fill, then pads it to a length drawn evenly from the band above the one its size falls in.	**4–13%**
Lane start	Sends each direction's first flight as the records Chrome and an HTTP/2 server open with.	**≈ 3.4 KB / lane**
Idle	Holds each WireGuard keepalive for a random interval, pads it to a length from the profile's idle sizes, then closes the lane and reopens when traffic resumes.	**Tiny**



Band edges are the profile's buckets: 16 bytes wide up to 256, 32 bytes wide above. A record's length tells an observer only which band it was in, every length in that band is equally likely, and none is under 9 bytes.

Figure 9. Band padding. Padding to a few fixed sizes would swap one pattern for another. Drawing evenly from the next band leaves WireGuard's fixed sizes sharing bands with ordinary packets: a lone handshake initiation comes out at the same lengths as a 144-byte packet.



Figure 10. Opening flight, record by record. Sizes are measured from Chromium loading a node's cover site. The web server's own `SETTINGS` frame (62 B on the wire) goes out before the node knows the connection is a lane. `HELLO` and `WELCOME` each straddle two records, because the frames are one byte stream cut at the profile's record boundaries.

Bulk records aren't padded. Their packets are full-size, and that size is no secret. But a record of whole full-size packets is that size times a count plus framing, a lattice nothing else on the network shows. Moving the record's end to a point drawn evenly inside its last packet (at least 64 bytes from either edge) spreads lengths evenly between one multiple and the next. It costs nothing except, sometimes, one more record.

Shaping profiles are data, not code. `WELCOME` names the profile to use, so the hub can roll out new profiles (bucket edges, idle sizes, first-flight record ranges, bulk threshold) without a protocol change. Two more rules keep control frames from standing out: a `PONG` or `RETIRE` sent alone is padded as an interactive record, and no record is padded to less than HTTP/2's smallest frame.

9 SNI modes and carriers

Networks filter on the handshake's server name in different ways, so the SNI is a per-network setting rather than one fixed value. Because the login happens *inside* TLS, **the node accepts any SNI, or none at all**.



On a new network the modes are tried in order, the best disguise first. The next one starts only after a failure, or after 2–5 s of silence. The first to log in is remembered for that network.

Figure 11. SNI modes. Trying the modes in order means a network where Own works never sees a borrowed name or a nameless handshake. The extra wait is paid once per network, and only where a filter drops packets without answering.

- **Certificate choice.** For its own domain the node presents the real certificate. For any other name, or none, it presents a default certificate, self-signed for `localhost` unless configured otherwise, the way most web servers behave. It never carries the machine's hostname.
- **Own-mode domains** carry no link to PangeaVPN: no brand name, private WHOIS, and a different registrar and DNS host. Wildcard certificates keep per-node subdomains out of Certificate Transparency logs, and a few base domains with a subdomain per node let the hub move nodes when one domain is blocked.

9.1 Carriers

The core (login, frames, scheduler and shaper) doesn't care how a lane reaches the node.

Carrier	What a lane is	Used when
Direct	A TLS 1.3 connection to the node on port 443, in one of the SNI modes.	By default. This is the fastest route.
CDN	A WebSocket over TLS to a CDN edge, which forwards it to the node. `HELLO` goes in the first WebSocket message, and the node also checks a header signed by the CDN.	Direct lanes can't open, for example because the node's IP is null-routed.

The client switches carriers within the same session, and direct and CDN lanes can run side by side during the switch. The user sees one transport that keeps working.

10 Threat model

Attack	Tectonic's answer	What's left
Passive DPI on the handshake	Chrome's ClientHello via uTLS, updated where uTLS lags behind Chrome (GREASE and ML-DSA signature algorithms), and checked field by field against Brave on Chromium 157. In Own mode, SNI, IP and certificate all agree.	uTLS, and Tectonic's own ClientHello spec, have to keep up with Chrome releases.
Active probing	Every connection belongs to the cover site's Go web server until a valid login claims it. Probes get `SETTINGS` first, the site, the same timeouts and the same close.	The cover site must look real and maintained. What a probe sees is a Go web server, which is itself a recognisable kind.
Replaying a login	Bound to the TLS exporter, and checked against a ±120 s timestamp window.	None known.
Long-connection heuristics	Lane churn limits each connection's lifetime to minutes.	Several connections from one client to one server is itself a pattern, though browsers open more than one too.
Per-connection throttling	Throttle detection adds lanes.	A per-host throttle, which caps all connections to one server together, can't be beaten this way.
Injected TCP resets	The lane is replaced and traffic continues on the others.	Resetting every lane at once forces a rejoin (≈ 2 RTT).
Size and timing analysis	The shaping budget hides WireGuard's fixed sizes and keepalive timer.	Machine-learning classifiers over whole flows.
SNI allowlist	Borrowed or None mode, chosen automatically and remembered per network.	A filter that checks names against IPs *and* drops connections with no SNI. The CDN carrier is next.
Node IP blocked	CDN carrier.	The CDN itself blocked; the cascade continues.
Cover domain blocked	The hub moves the node to a new domain.	Needs a supply of domains.
IP allowlist	Out of scope.	Every protocol fails here.

11 Compared with existing protocols

Each of these designs solves a real problem well. The comparison shows where Tectonic made different choices.

Protocol	What it does	How Tectonic differs
VLESS + REALITY	Borrows another site's TLS identity, and accepts only the names it's configured for.	Uses its own identity by default. When it borrows a name it accepts any name, or none. Traffic is spread across lanes.
Trojan	Real TLS plus a password, with a fallback website, over one long-lived stream.	The login is channel-bound, so it can't be replayed. Lanes churn, and datagrams aren't tied to one connection's order.
Cloak	Hides its login in ClientHello fields, and multiplexes ordered streams over several connections.	The ClientHello is unmodified Chrome. Datagrams, not streams, are merged, and the wait for a lost one is bounded.
AnyTLS	TLS with padding aimed at defeating TLS-in-TLS detection.	There is no TLS inside the TLS, because the payload is WireGuard. Shaping targets WireGuard's own patterns.
NaiveProxy	Chrome's real network stack and HTTP/2 CONNECT.	Pure Go with no native library, so it runs on iOS too. Multiple lanes and datagram-native framing.
Speed-first VPNs	Built for speed, with little or no disguise.	Speed comes from the scheduler, and the traffic still looks like browsing.

12 Evaluation

The `bench/` harness runs Tectonic, REALITY and AnyTLS models, and plain WireGuard over UDP, under identical conditions. The figures below come from the **kernel path**: three Linux network namespaces (client ↔ router ↔ server) joined by veth pairs, with netem for delay and loss, tbf for the bottleneck and a one-BDP queue, and fq `maxrate` for per-connection caps. WireGuard is wireguard-go on real TUN devices, so the TCP inside the tunnel is Linux cubic. This is the path that Linux, Android, macOS and iOS clients take. Measured 2026-11-10 on Go 1.27.3 and Linux 7.3.3, three passes for each node congestion control, reported as means.

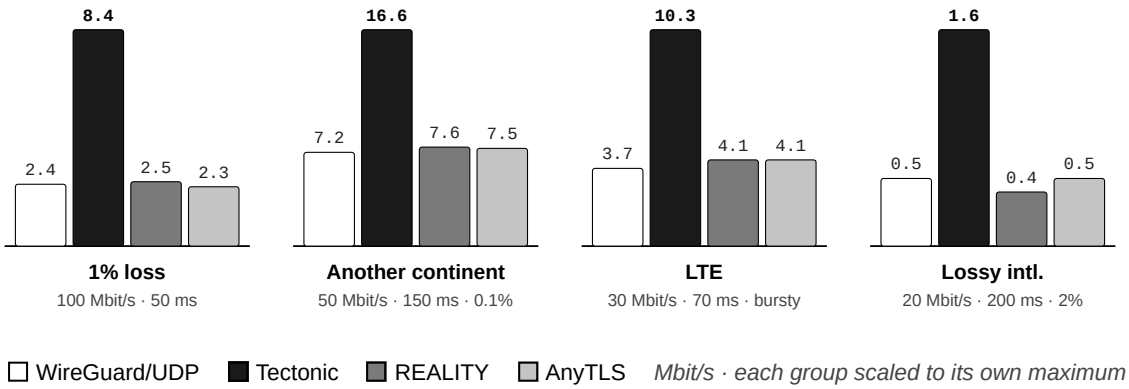


Figure 12. Throughput on lossy paths, node on cubic (Mbit/s). Each lane has its own congestion window, so random loss halves only part of the tunnel's sending rate. Tectonic carries 2.2–3.4× what the single-connection models do, and on these paths it beats plain WireGuard as well, because over UDP the inner TCP meets every loss itself, while each lane's TCP repairs losses before the tunnel sees them.

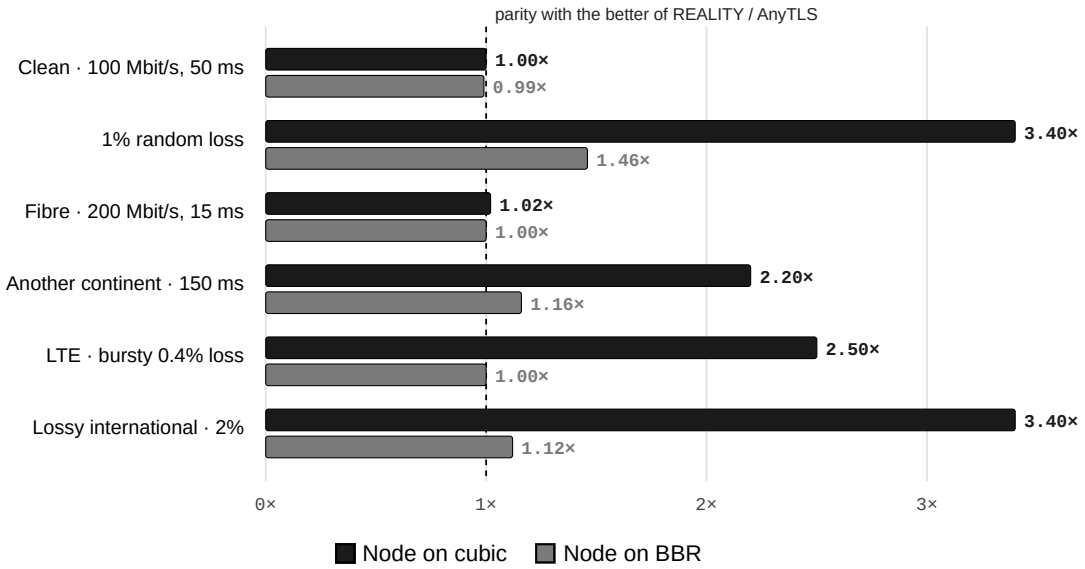


Figure 13. Tectonic throughput relative to the better of REALITY and AnyTLS. With a BBR node, every transport recovers better from loss, so the gap narrows to 1.1–1.5× on three lossy paths. Clean links are level throughout. The one case below parity (clean link, BBR node) is 1% behind REALITY and level with AnyTLS.

12.1 Absolute throughput (Mbit/s, mean of three passes)

Path	WireGuard	Tectonic	REALITY	AnyTLS	Tectonic (BBR)	Best model (BBR)
Clean · 100 Mbit/s, 50 ms	91.6	**88.1**	87.8	87.4	**83.4**	84.3
1% loss · 100 Mbit/s, 50 ms	2.4	**8.4**	2.5	2.3	**79.2**	54.1
Fibre · 200 Mbit/s, 15 ms	183.1	**178.7**	175.8	175.0	**172.5**	172.1
Another continent · 50 Mbit/s, 150 ms, 0.1%	7.2	**16.6**	7.6	7.5	**37.4**	32.3
LTE · 30 Mbit/s, 70 ms, bursty 0.4%	3.7	**10.3**	4.1	4.1	**22.2**	22.2
Lossy international · 20 Mbit/s, 200 ms, 2%	0.5	**1.6**	0.4	0.5	**6.7**	6.0

Columns 2–5: node on cubic. WireGuard runs over UDP, so the node's TCP congestion control doesn't affect it.

12.2 Latency under load

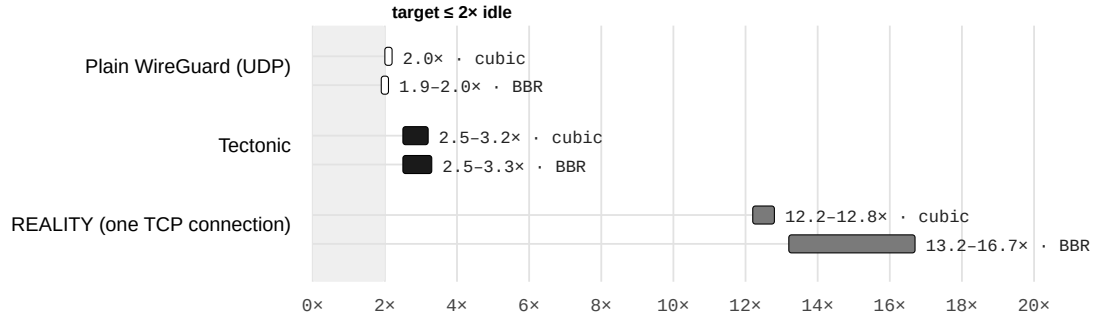


Figure 14. p95 ping inside the tunnel during a saturating download, as a multiple of idle RTT (20 Mbit/s, 50 ms). Small packets overtake bulk data and records are kept short, so Tectonic stays within a few times idle RTT while a single TLS connection reaches 12–17×. It is still above the 2× target, most likely because pings wait behind the path's own queue and the 32 KB a lane's kernel may hold.

12.3 Design targets

Target	Simulator	Kernel · cubic	Kernel · BBR
Time to first packet ≤ 2 RTT	✓ 2.05–2.07	✓ 2.03–2.08	✓ 2.03–2.04
Clean link ≥ 90% of WireGuard	✓ 99–114%	✓ 95–97%	✓ 91%
Bulk overhead ≤ 3%	~ on the line	~ 2.7–3.1%	~ 2.9–3.3%
1% loss: clearly above REALITY / AnyTLS	✓ 2.9–3.9×	✓ 3.1–3.7×	✓ 1.4–1.5×
Rises with lanes under 4 Mbit/s per-conn cap	✓ 21–29 Mbit/s	✓ 19–24 Mbit/s	✓ 10–17 Mbit/s
Latency under load ≤ 2× idle	✗ 2.0–2.9×	✗ 2.5–3.2×	✗ 2.5–3.3×
No visible stall after a lane reset	✓ ≤ 76 ms	✓ 107–139 ms	✓ 102–130 ms
Interactive overhead ≤ 15%	✓ 5.7–5.8%	✓ 4.5–5.1%	✓ 12.3–13.0%

Ranges are over passes, and a ✓ needs every pass to meet the target. A later round of fingerprint hardening lowered interactive overhead further, to 4.2% (simulator), 4.0% (cubic) and 10.6% (BBR), averaged over three passes.

12.4 Fingerprint audit

A fingerprint suite captures Tectonic's sessions (idle, interactive, browsing, download and upload), compares them with captures of Brave 157 (Chromium), and sends 20 kinds of active probe to a node and to a stock Go HTTPS server. The table compares the protocol before and after the November 2026 hardening round.

Measure	Before	After	Reference
Active probes answered like a stock Go server	13–14 / 20	**19–20 / 20**	20 / 20
Server-closed probes ending with a TLS alert	16 / 20	**20 / 20**	stock Go TLS
Client's first record	405–612 B	**87 B**	Brave: 87 B
Server's first record	1296–4191 B	**62 B**	HTTP/2 SETTINGS: 62 B
Lanes whose first record matches Chrome	0 / 30	**24 / 24**	—
Download records on a `k·1455+17` lattice	96%	**0%**	—
Distinct download record lengths	48	**557**	—
Browsing: share of the three most common lengths	69%	**7%**	—
Idle lane close times	20.0–20.4 s	**17–99 s**	random
Interactive wire bytes	230 KB	**206 KB**	−10.5%

Two probe types still differ from a stock server: a 24-byte garbage write on h2, and an 81-byte HELLO-shaped record. Idle sessions use 27% more wire bytes than before, the cost of replacing a fixed 20 s turnover with randomised lane churn. Since this audit, the ClientHello's `signature_algorithms` list has gained the GREASE value and three ML-DSA schemes that Chrome has sent since uTLS's last Chrome profile (133), and bulk records now end inside their last packet, which removes the remaining packet-size lattice.

13 Implementation and deployment

Tectonic is about 20,000 lines of Go, with client and node in one module. The client is a library that the PangeaVPN daemon links on desktop, Android and iOS. The node is a single binary.

Package	Responsibility
`wire/`	Frames, HELLO/WELCOME and key derivation. No I/O; unit- and fuzz-tested.

Package	Responsibility
`lane/`	One TLS lane: dial, login, frame I/O and retire. Direct and WebSocket (CDN) carriers, plus per-OS socket state (`TCP_INFO`, `TCP_NOTSENT_LOWAT`).
`sched/`	Lane pool, scheduler, resequencer, duplication, churn, throttle detection and send limit.
`shape/`	Shaping profiles and record-size planning.
`server/`	TLS listener, login check, cover-site fallback, session table and UDP forwarding to WireGuard.
`client.go`	Public API: dial a profile, get a `net.PacketConn` for WireGuard.
`cmd/tectonic-server/`	Node binary: `serve`, `pin` and `issue`; SIGHUP reloads certificates and the revocation list.
`bench/`	Simulated-link (gVisor) and kernel-namespace benchmarks against REALITY, AnyTLS and WireGuard.

```
conn, err := tectonic.Dial(ctx, tectonic.Config{
    Profile: profile,   // node, cover_domain, key_id, device_key, pins, cdn...
    Network: networkID, // the SNI mode that works is remembered per network
    Control: protect,   // optional: keep lane sockets out of the tunnel on mobile
})
err = conn.WaitForSession(ctx, 15*time.Second)
```

Nodes are built with Go 1.27 or newer, for byte-identical web-server behaviour. Raise `net.core.rmem_max` so that the node's 7 MB WireGuard socket buffer can absorb WireGuard's bursts.

14 Status, limitations and roadmap

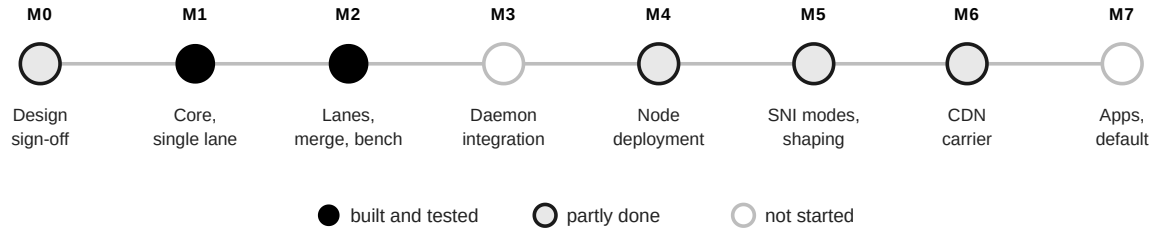


Figure 15. Milestones. The protocol library, node binary, SNI modes, shaping and CDN carrier code are in this repository. Still to come: open design questions (M0), daemon integration (M3), node rollout and hub credential issuance (M4), shaping profiles recorded from real Chrome sessions (M5), CDN deployment (M6), and field data before Tectonic becomes the cascade default (M7).

14.1 Known limitations

- **Latency under load** is 2.5–3.3× idle on the kernel path, above the 2× target, though far below single-connection designs.
- **Per-connection cap with a BBR node.** The session send limit costs throughput here (10–17 against 17–27 Mbit/s without it). The fix needs a ceiling for each lane, and the four approaches tried so far each cost 2–10% elsewhere.
- **Bulk overhead** sits on the 3% line, about 6 points of which are TCP's own headers and ACKs.
- **Shaping profiles** are measured from Chromium for the opening flight only. The other numbers are placeholders until profiles are recorded from real Chrome sessions.
- **Platform coverage of the send limit.** It reads Linux's `TCP_INFO`, so it is off on other client platforms. This matters only for uploads.

- **Flow-level ML classifiers** remain a risk that no amount of shaping rules out. That is why the transport cascade stays in place.

14.2 Open questions

1. **Domains:** one base domain per group of nodes, and who registers and rotates them.
2. **Node deployment:** who deploys `tectonic-server`, and whether port 443 is free on every node.
3. **Hub changes:** holding `node_secrets`, issuing credentials, and a revocation list.
4. **Custom SNI:** which allowed names the hub should supply by default.
5. **Mobile:** the right lane count on cellular, and the battery cost of lane churn.

15 Conclusion

Tectonic builds a censorship-resistant transport around one fact: its payload is always WireGuard datagrams. That lets it use several short-lived, independently congestion-controlled TLS lanes that each look like Chrome loading a website, with no cryptography of its own. In the lab it carries 2–3× what single-connection TLS transports carry on lossy paths, matches them on clean ones, starts in two round trips, and answers probes the way an ordinary Go web server does. The next step is field data.

References

1. J. A. Donenfeld. *WireGuard: Next Generation Kernel Network Tunnel.* NDSS, 2017.
2. E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3.* RFC 8446, 2018.
3. E. Rescorla. *Keying Material Exporters for Transport Layer Security (TLS).* RFC 5705, 2010.
4. H. Krawczyk and P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF).* RFC 5869, 2010.
5. M. Thomson and C. Benfield. *HTTP/2.* RFC 9113, 2022.
6. S. Frolov and E. Wustrow. *The use of TLS in Censorship Circumvention.* NDSS, 2019. (uTLS)
7. S. Ha and I. Rhee. *Taming the Elephants: New TCP Slow Start.* Computer Networks 55(9), 2011. (HyStart)
8. N. Cardwell et al. *BBR: Congestion-Based Congestion Control.* ACM Queue 14(5), 2016.
9. Project X. *XTLS/REALITY.* Software, github.com/XTLS/REALITY.
10. PangeaVPN. *Tectonic design methodology and benchmark results.* docs/design.md, bench/RESULTS.md, bench/recent.md.